



**SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES**



ASSESSMENT TOOL 1

Experiments

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1104

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. **(BL4)**

Assessment Tool Weightage: **40%**

QUESTIONS: 4

Total Marks: 40

Code of Conduct:

I **S.poojitha(192320076)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Poojitha

Reg. No: 192320076

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Book Bank Process

Problem Statement: Develop a system using UML for implementing the Book Bank process. It should verify the details of the student by the central computer. The details regarding the student will be provided to the central computer through the administrator in the book bank and the computer will verify the details of the student and provide approval to the office. Then the books that are needed by the student will issue from the office to him.

1. Actors Identified

- Student — requests books and receives approval status
- Administrator — submits student details to the central computer
- Central Computer — verifies student details and issues approval
- Book Bank Office — issues books once approval is received

2. Use Case Diagram

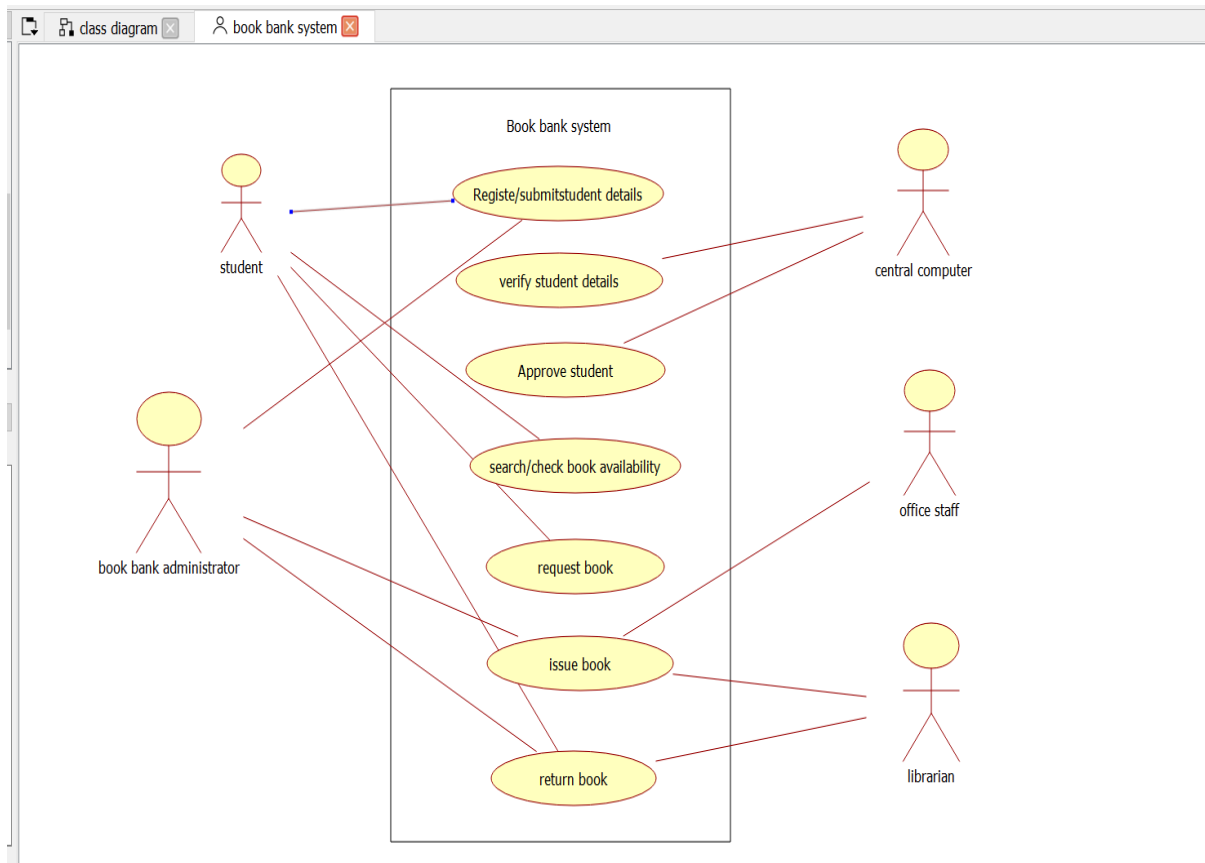


Figure 1.1 — Use case diagram for the Book Bank System

3. Class Diagram

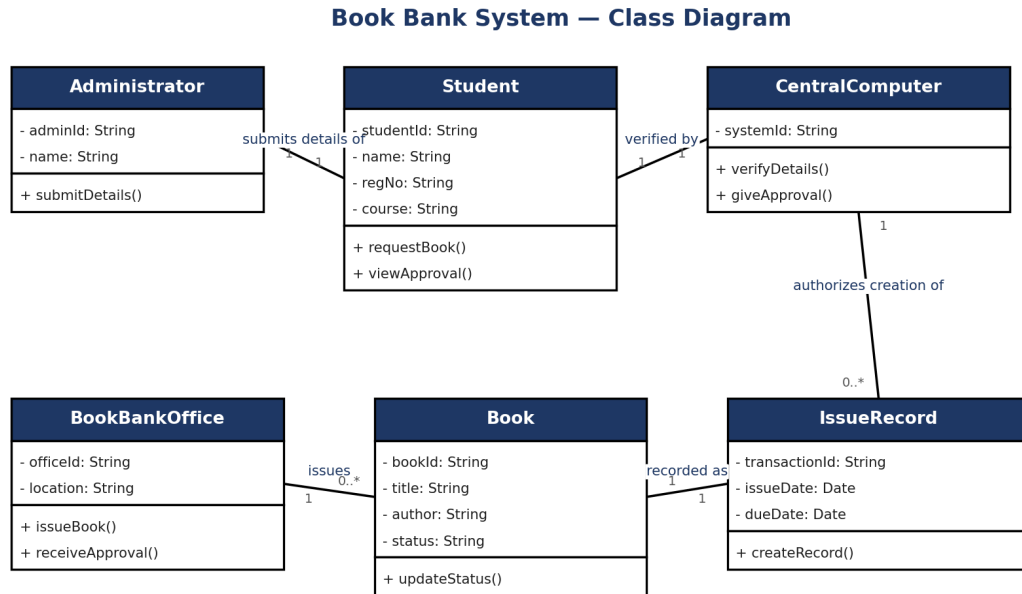


Figure 1.2 — Class diagram for the Book Bank System

4. Interaction Flow

1. The Administrator collects and submits the student's details to the Central Computer.
2. The Central Computer verifies the student's details against its stored records.
3. On successful verification, the Central Computer sends an approval to the Book Bank Office.
4. The Student requests the required books at the Book Bank Office.
5. The Office checks that approval has been received and issues the requested books.
6. An IssueRecord is created, capturing the transaction, issue date, and due date.

5. Design Analysis

The use case diagram separates the four actors' distinct goals — data submission (Administrator), verification (Central Computer), book issuing (Office), and book requesting (Student) — connected through `<<include>>` relationships that enforce the mandatory verify-then-approve-then-issue sequence described in the problem statement. The class diagram reflects the same separation of responsibility at the data level: CentralComputer is solely responsible for `verifyDetails()` and `giveApproval()`, while BookBankOffice is solely responsible for `issueBook()`, preventing any single class from taking on more than one concern. The Student–CentralComputer and CentralComputer–IssueRecord associations together enforce that a book can only be recorded as issued after verification has occurred, which directly satisfies the requirement that the central computer's approval must precede issuing. This design is also readily extensible — a future "Return Book" use case could be added without modifying the verification logic, since it only concerns the Book and IssueRecord classes.

Experiment 2: Exam Registration Portal

Problem Statement: Develop a system using UML for implementing the Exam Registration Portal. It should verify the details of the candidate by the central computer. The details regarding the candidate will be provided to the central computer through the administrator and the computer will verify the details of the candidate and provide approval. Then the hall ticket will be issued from the office to the candidate.

1. Actors Identified

- Candidate — registers for the exam and receives the hall ticket
- Administrator — submits candidate details to the central computer
- Central Computer — verifies candidate details and issues approval
- Exam Office — issues the hall ticket once approval is received

2. Use Case Diagram

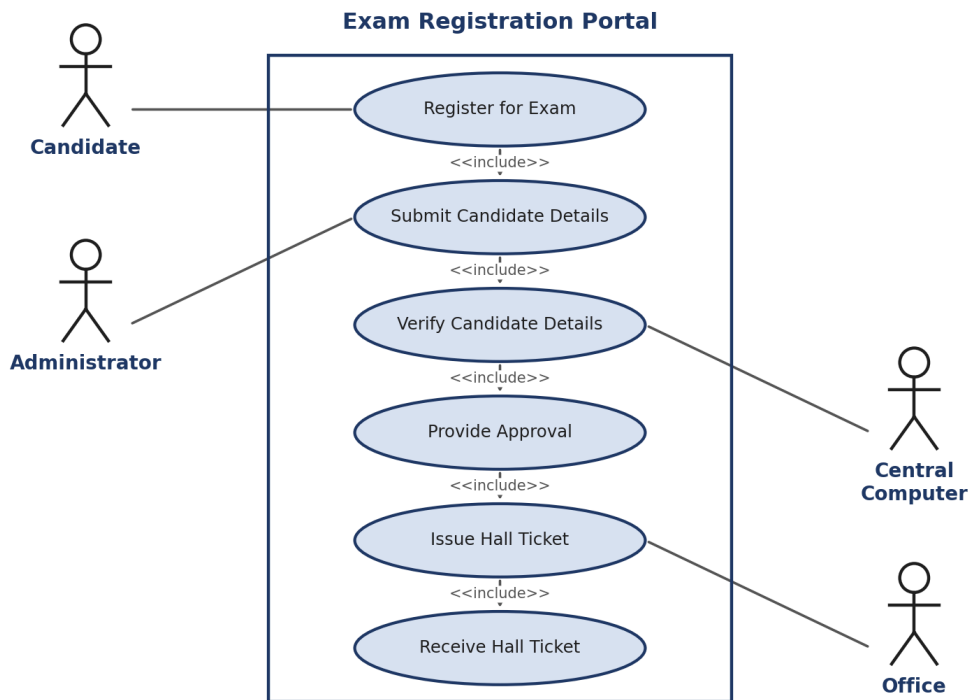


Figure 2.1 — Use case diagram for the Exam Registration Portal

3. Class Diagram

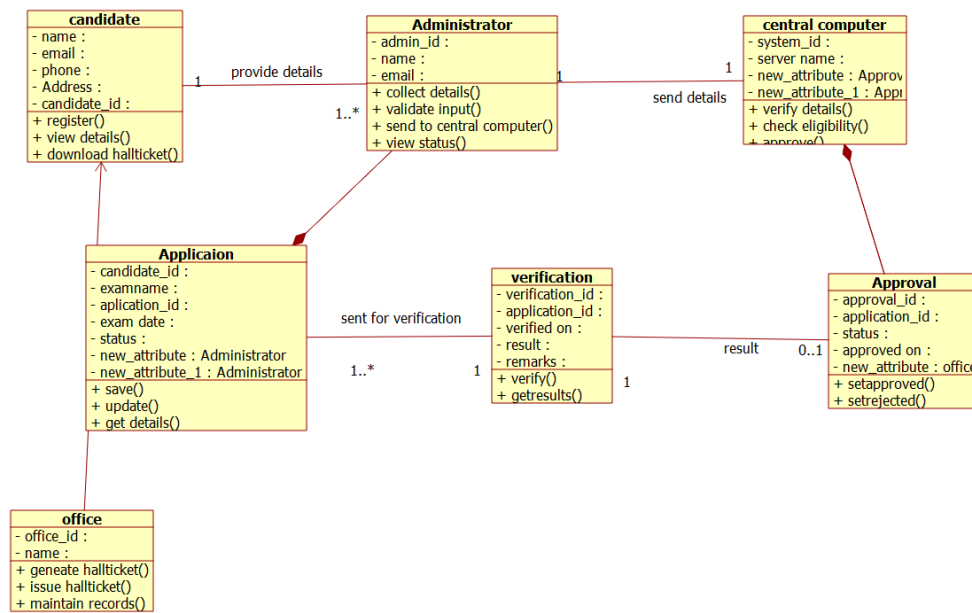


Figure 2.2 — Class diagram for the Exam Registration Portal

4. Interaction Flow

1. The Candidate registers for the exam, providing personal and exam-related details.
2. The Administrator submits the candidate's details to the Central Computer for verification.
3. The Central Computer verifies the candidate's details against its records.
4. On successful verification, the Central Computer provides approval, authorizing the registration.
5. The Exam Office generates and issues the hall ticket for the candidate.
6. The Candidate receives the hall ticket, confirming successful exam registration.

5. Design Analysis

This design mirrors the Book Bank process structurally (submit → verify → approve → issue), reflecting that both are instances of a general "verified issuance" workflow — a useful pattern to recognise when analysing similar requirements. The ExamRegistration class captures the state of a candidate's registration (status) independently of the HallTicket itself, which allows the system to represent a candidate who is registered but not yet issued a ticket, an important intermediate state that a simpler design conflating the two classes would lose. The CentralComputer–ExamRegistration association models that only an authorized verification can move a registration into an approved state, directly satisfying the requirement that the computer's approval must occur before the hall ticket is issued.

Experiment 3: Stock Maintenance System

Problem Statement: Develop a system using UML for implementing Stock Maintenance System. In this system, the customer can place orders and purchase items with the aid of the stock dealer and central stock system. These orders are verified and the items are delivered to the customer. Then the stock details are to be updated accordingly.

1. Actors Identified

- Customer — places orders and receives delivered items
- Stock Dealer — verifies orders and arranges delivery
- Central Stock System — cross-checks stock levels and updates stock records

2. Use Case Diagram

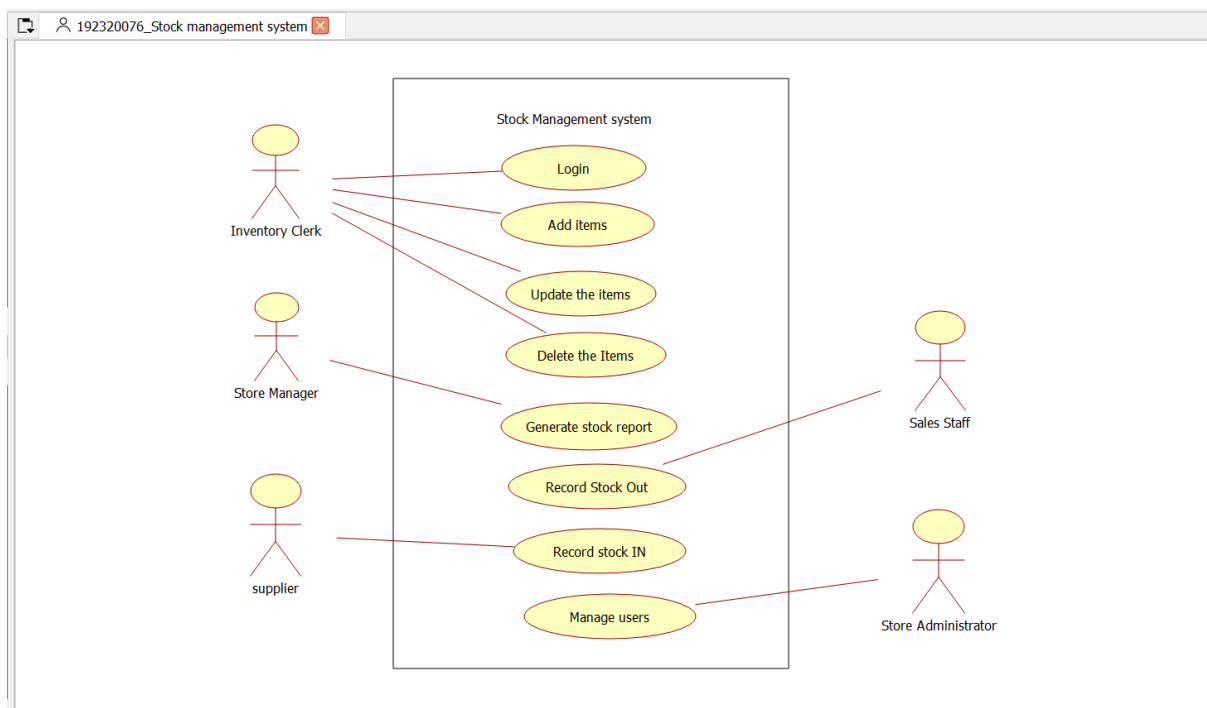


Figure 3.1 — Use case diagram for the Stock Maintenance System

3. Class Diagram

Stock Maintenance System — Class Diagram

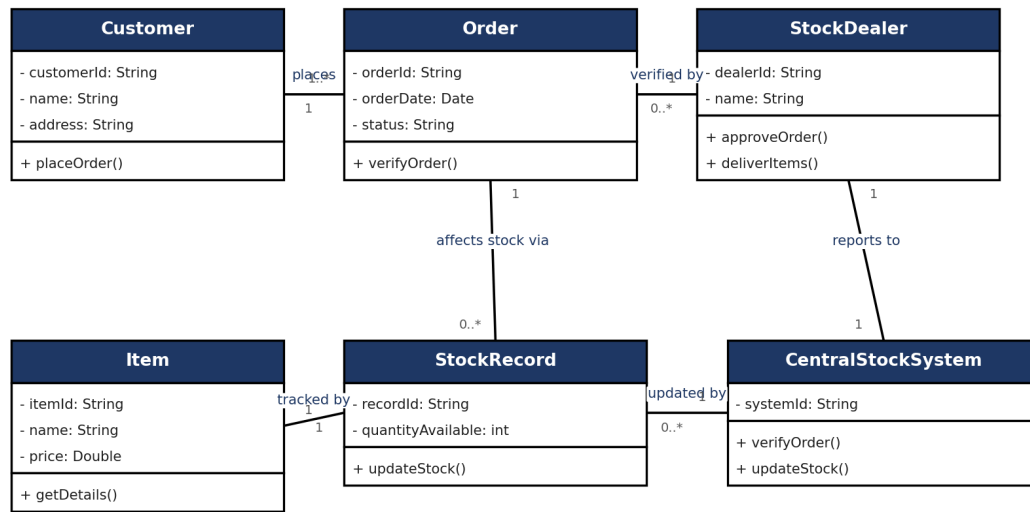


Figure 3.2 — Class diagram for the Stock Maintenance System

4. Interaction Flow

1. The Customer places an order for one or more items.
2. The Stock Dealer verifies the order against item availability and customer details.
3. The Central Stock System cross-checks current stock levels for the ordered items.
4. Once verified, the Stock Dealer arranges delivery of the items to the customer.
5. The Customer receives the ordered items.
6. The Central Stock System updates the stock records to reflect the reduced quantity.

5. Design Analysis

Unlike the first two experiments, this system involves a genuine many-to-many relationship between orders and items, modelled here by the Order–StockRecord and Item–StockRecord associations rather than a single linear chain — reflecting that a real order can contain multiple items, and a single item's stock is affected by many orders over time. Separating Item (the catalogue definition of a product) from StockRecord (the current quantity available) follows good object-oriented practice: it lets the catalogue information remain stable while `quantityAvailable` changes frequently, and it lets `updateStock()` be implemented and tested independently of the product catalogue. The StockDealer–CentralStockSystem association ensures that stock updates are always routed through the authoritative CentralStockSystem, preventing inconsistent stock counts if multiple dealers were to operate concurrently.

Experiment 4: Online Course Reservation System

Problem Statement: Develop a system using UML for implementing Online Course Reservation System. This system is organized by the central management system. The student first browses and selects the desired course of their choice. The university then checks the availability of the seat if it is available the student is enrolled for the course.

1. Actors Identified

- Student — browses, selects courses, and views enrollment status
- Central Management System — checks seat availability and enrolls students

2. Use Case Diagram

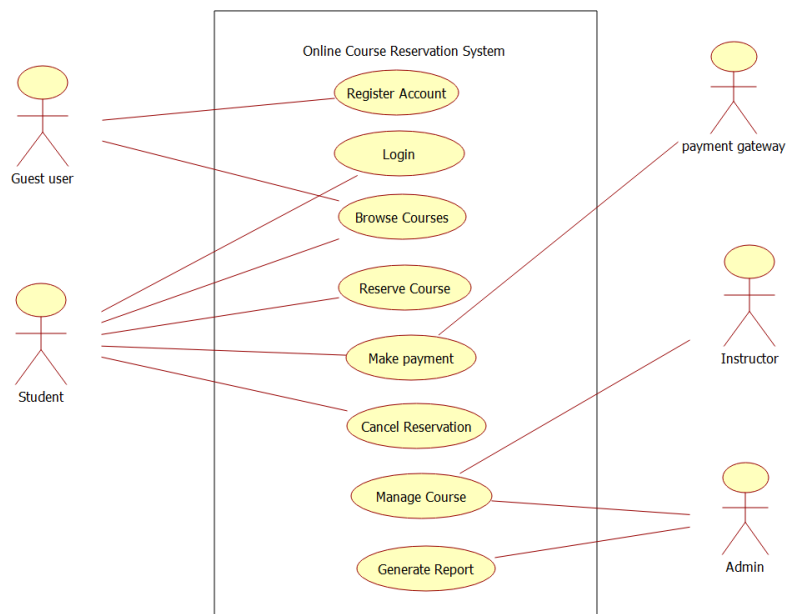


Figure 4.1 — Use case diagram for the Online Course Reservation System

3. Class Diagram

Online Course Reservation System — Class Diagram

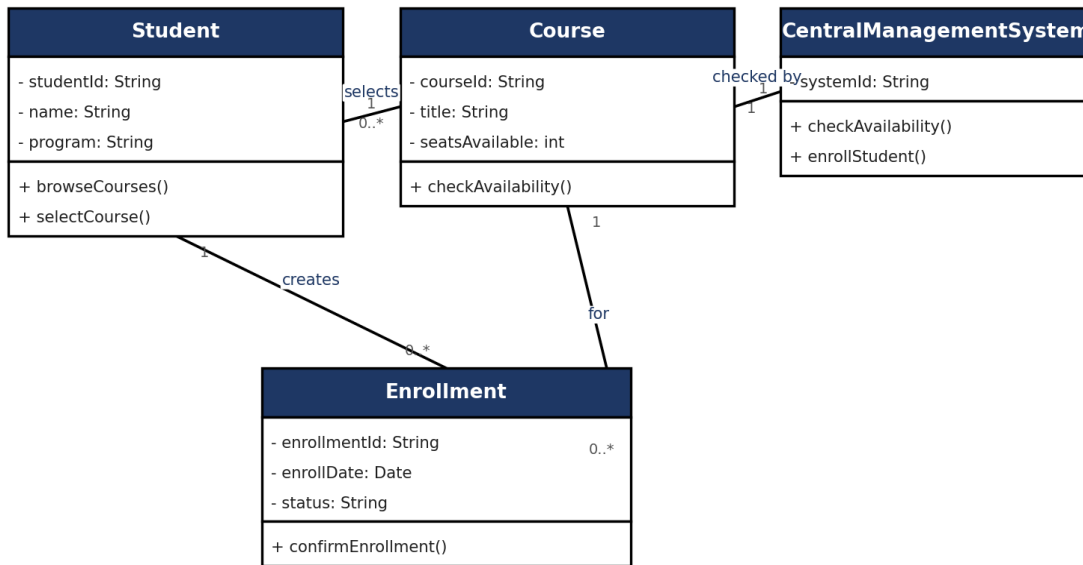


Figure 4.2 — Class diagram for the Online Course Reservation System

4. Interaction Flow

1. The Student browses the list of available courses through the portal.
2. The Student selects a desired course.
3. The Central Management System checks seat availability for the selected course.
4. If a seat is available, the system enrolls the Student in the course and creates an Enrollment record.
5. If no seat is available, the <<extend>> path notifies the Student instead of creating an enrollment.
6. The Student can view their current enrollment status at any time.

5. Design Analysis

This system has the simplest actor set of the four (only two actors), which is reflected in its use case diagram by a shorter <<include>> chain and a single <<extend>> relationship modelling the conditional "no seat available" outcome — an appropriate use of <<extend>> rather than <<include>>, since checking availability does not always lead to enrollment. The Enrollment class is deliberately kept separate from both Student and Course rather than folding seat-tracking directly into Course, because a course's seatsAvailable attribute changes with every enrollment while the Enrollment record itself preserves a permanent history of who enrolled and when — an important distinction for CO3's emphasis on correctly identifying classes and attributes rather than merging unrelated concerns into a single class.